

アカウント一覧参照・一部編集・削除申請承認

ここからは、「【管理者用】各種管理画面（アカウント一覧参照編集・削除申請の承認）」の実装フェーズへとステップアップしましょう。

管理者が一覧から「退会申請中（`quitDemand === 1`）」のユーザーを視覚的なバッジ表示で見つけ出し、「承認ボタン」をワンクリックすることでデータベースから物理削除、あるいはステータスを完全更新する機能を構築します。

バックエンド側の実装

UserAccountService.java への処理追加

管理者による「退会承認（アカウントの削除処理）」を実行するためのロジックをサービス層に追加します。

今回は仕様書の「物理削除」の文脈に合わせ、指定された `accountId` のレコードをデータベースから完全に削除するロジックを実装します。

`UserAccountService.java` の末尾（クラスの閉じ括弧 `}` の直前）に、以下を追加してください。

```
/**
 * 管理者による退会申請の承認処理（アカウントの削除）
 * @param accountId 削除対象のアカウントID
 */
public void approveQuitDemand(String accountId) {
    // 1. 対象のユーザーが本当に存在するか確認
    UserAccount user = repository.findById(accountId)
        .orElseThrow(() -> new RuntimeException("対象のユーザーが見つかりません。"));
}
```

```
// 2. セーフティガード：万が一、統括管理者を削除しようとした場合は阻止
if ("admin@example.com".equals(user.getUserId())) {
    throw new RuntimeException("統括管理者アカウントは削除できません。");
}
// 3. データベースからアカウントを物理削除
repository.delete(user);
System.out.println("=== [System] 管理者によってアカウントが削除されました: " + user.getUserId() + " ===");
}
```

UserAccountController.java へのエンドポイント追加

フロントエンド（Vue.js）からのリクエストを受け付ける窓口を作成します。

- アカウント一覧を取得するAPI（※既存の `/debug-list` を管理者用に正式なエンドポイントとして綺麗に整備します）
- 退会申請を承認（削除実行）するAPI（`DELETE /api/users/approve-quit/{accountId}`）

`UserAccountController.java` の末尾（クラスの閉じ括弧 `}` の直前）に、以下の2つのメソッドを追加してください。

```

/**
 * 管理者用：すべてのアカウント一覧を取得するエンドポイント
 * GET http://localhost:8080/api/users/admin/list
 */
@GetMapping("/admin/list")
public ResponseEntity<List<UserAccount>> getAdminUserList() {
    // サービス層から全ユーザーを取得して返却
    List<UserAccount> users = userService.findAll();
    return ResponseEntity.ok(users);
}

/**
 * 管理者用：退会申請を承認（アカウント削除）するエンドポイント
 * DELETE http://localhost:8080/api/users/approve-quit/{accountId}
 */
@DeleteMapping("/approve-quit/{accountId}")
public ResponseEntity<String> approveQuit(@PathVariable String accountId) {
    try {
        System.out.println("--- 退会承認リクエストを受信 ---");
        System.out.println("対象accountId: " + accountId);

        // サービス層の削除処理を呼び出し
        userService.approveQuitDemand(accountId);

        return ResponseEntity.ok("退会申請を承認し、アカウントを削除しました。");
    } catch (RuntimeException e) {
        // エラー時は 400 Bad Request を返却
        return ResponseEntity.badRequest().body("承認処理に失敗しました: " + e.getMessage());
    }
}

```

フロントエンド側の実装

この画面では以下を実現します。

1. すべてのアカウント一覧をテーブル（表）形式で参照できること
2. 退会申請中のユーザー（ `quitDemand === 1` ）に視覚的な「バッジ」を表示すること
3. バッジのあるユーザーの横に「承認する」ボタンを配置し、クリックして確認後に安全に物理削除（バックエンドの `DELETE` APIをコール）できること

フロントエンド画面 `AdminUserListView.vue` の作成

`src/views`（またはお使いのViewコンポーネント配置フォルダ）配下に、新しく `AdminUserListView.vue` というファイルを作成し、以下の完全版ソースコードを貼り付けてください。

```
<template>
  <div class="admin-container">
    <div class="admin-header-box">
      <h2>管理者用：アカウント一覧・退会管理</h2>
      <p class="admin-subtitle">登録されているユーザーの確認、および退会申請の承認（アカウント削除）を行えます。</p>
    </div>
```

```

<div v-if="isLoading" class="loading-box">
  データを読み込み中...
</div>
<div v-if="errorMessage" class="alert alert-danger">{{ errorMessage }}</div>
<div v-if="successMessage" class="alert alert-success">{{ successMessage }}</div>

<div v-if="!isLoading && users.length > 0" class="table-responsive">
  <table class="user-table">
    <thead>
      <tr>
        <th>ユーザー名</th>
        <th>ユーザーID (メールアドレス)</th>
        <th>権限</th>
        <th>ステータス</th>
        <th>操作</th>
      </tr>
    </thead>
    <tbody>
      <tr v-for="user in users" :key="user.accountId" :class="{ 'row-quit-pending': user.quitDemand === 1 }">
        <td><strong>{{ user.userName }}</strong></td>

        <td>{{ user.userId }}</td>

        <td>
          <span :class="['badge', user.isAuth === 1 ? 'badge-admin' : 'badge-general']">
            {{ user.isAuth === 1 ? '統括管理者' : '一般ユーザー' }}
          </span>
        </td>
        <td>
          <span v-if="user.quitDemand === 1" class="badge badge-warning animate-pulse">
            ⚠ 退会申請中
          </span>
          <span v-else class="badge badge-normal">
            正常稼働
          </span>
        </td>
      </tr>
    </tbody>
  </table>
</div>

```

```

    <td>
      <button
        v-if="user.quitDemand === 1 && user.userId !== 'admin@example.com'"
        @click="handleApproveQuit(user)"
        class="btn-approve"
      >
        退会を承認する
      </button>
      <span v-else-if="user.userId === 'admin@example.com'" class="text-muted-info">-</span>
      <span v-else class="text-muted">-</span>
    </td>
  </tr>
</tbody>
</table>
</div>

<div v-else-if="!isLoading" class="empty-box">
  表示できるアカウントが存在しません。
</div>
</div>
</template>
<script setup>
import { ref, onMounted } from 'vue';
import { useRouter } from 'vue-router';
import apiClient from '../api'; // 共通APIクライアント (/api がベースURLの前提)

const router = useRouter();

const users = ref([]);
const isLoading = ref(true);
const errorMessage = ref('');
const successMessage = ref('');

```

```
// 1. 全アカウント一覧を取得する関数
const fetchUserList = async () => {
  try {
    errorMessage.value = '';
    // 先ほどJava側で作成した GET /api/users/admin/list を叩く (/apiを除いたパス)
    const response = await apiClient.get('/users/admin/list');
    users.value = response.data;
  } catch (error) {
    console.error(error);
    errorMessage.value = 'アカウント一覧の取得に失敗しました。管理者権限をご確認ください。';
  } finally {
    isLoading.value = false;
  }
};

// 画面表示時に実行（ガードチェックも兼ねる）
onMounted(() => {
  const userData = localStorage.getItem('user');
  if (userData) {
    const user = JSON.parse(userData);
    // 統括管理者（isAuth === 1）でなければ、不正アクセスとしてダッシュボードへ強制送還
    if (user.isAuth !== 1) {
      alert('この画面は管理者専用です。');
      router.push('/dashboard');
      return;
    }
    // 管理者であれば一覧を取得
    fetchUserList();
  } else {
    router.push('/login');
  }
});
```



```
// 2. 退会申請を承認（アカウント削除）する処理
const handleApproveQuit = async (targetUser) => {
  successMessage.value = '';
  errorMessage.value = '';

  // 仕様に基づく最終確認アラート
  const confirmAction = confirm(`【最終警告】 \n本当に「${targetUser.userName}」さんの退会申請を承認しますか？ \nこの操作を行うと、アカウント情報はシステムから完全に物理削除されます。`);
  if (!confirmAction) return;

  try {
    // 先ほどJava側で作成した DELETE /api/users/approve-quit/{accountId} を叩く
    await apiClient.delete(`/users/approve-quit/${targetUser.accountId}`);

    // 成功メッセージを表示
    successMessage.value = `「${targetUser.userName}」さんのアカウント削除（退会承認）が完了しました。`;

    // 一覧リストを最新の状態に再リロード
    await fetchUserList();
  } catch (error) {
    if (error.response && error.response.data) {
      errorMessage.value = error.response.data;
    } else {
      errorMessage.value = '承認処理の通信中にエラーが発生しました。';
    }
  }
};
</script>

<style scoped>
.admin-container {
  max-width: 1000px;
  margin: 30px auto;
  padding: 20px;
}
```

```
.admin-header-box {
  background: #ffffff;
  padding: 20px;
  border-radius: 8px;
  box-shadow: 0 2px 8px rgba(0,0,0,0.05);
  margin-bottom: 25px;
  border-left: 5px solid #dc3545; /* 管理者カラーの赤 */
}
h2 {
  margin: 0 0 5px 0;
  color: #333;
}
.admin-subtitle {
  margin: 0;
  color: #6c757d;
  font-size: 0.95rem;
}

/* テーブル周りのデザイン */
.table-responsive {
  background: #ffffff;
  border-radius: 8px;
  box-shadow: 0 4px 12px rgba(0,0,0,0.08);
  overflow: hidden;
}
.user-table {
  width: 100%;
  border-collapse: collapse;
  text-align: left;
}
.user-table th, .user-table td {
  padding: 15px 20px;
  border-bottom: 1px solid #dee2e6;
}
.user-table th {
  background-color: #f8f9fa;
  color: #495057;
  font-weight: bold;
}
```

```
/* 退会申請中の行をほんのり薄黄色にして目立たせる */
.row-quit-pending {
  background-color: #fffd5;
}
/* 各種バッジスタイル */
.badge {
  display: inline-block;
  padding: 5px 10px;
  font-size: 0.8rem;
  font-weight: bold;
  border-radius: 20px;
}
.badge-admin { background-color: #e3f2fd; color: #0d47a1; }
.badge-general { background-color: #e8f5e9; color: #1b5e20; }
.badge-normal { background-color: #f1f3f5; color: #6c757d; }
.badge-warning { background-color: #fff3cd; color: #856404; border: 1px solid #ffeeba; }

/* 点滅アニメーション（視覚的ギミック） */
@keyframes pulse {
  0% { opacity: 1; }
  50% { opacity: 0.6; }
  100% { opacity: 1; }
}
.animate-pulse {
  animation: pulse 2s infinite;
}

/* 承認ボタンのスタイル */
.btn-approve {
  background-color: #dc3545;
  color: white;
  border: none;
  padding: 8px 14px;
  border-radius: 4px;
  font-size: 0.85rem;
  font-weight: bold;
  cursor: pointer;
  transition: background-color 0.15s;
}
.btn-approve:hover {
  background-color: #bd2130;
}
```

```
.text-muted { color: #ced4da; }
.text-muted-info { color: #adb5bd; font-size: 0.85rem; font-style: italic; }

/* メッセージボックス */
.alert {
  padding: 15px;
  margin-bottom: 20px;
  border-radius: 4px;
  font-weight: bold;
}
.alert-danger { background-color: #f8d7da; color: #721c24; border-left: 5px solid #dc3545; }
.alert-success { background-color: #d4edda; color: #155724; border-left: 5px solid #28a745; }
.loading-box, .empty-box { text-align: center; padding: 40px; color: #6c757d; font-size: 1.1rem; }
</style>
```

ルーターへの登録（router/index.js 等）

作成した画面へブラウザからアクセス、またはハンバーガーメニューから遷移できるように、Vue Router（ルーティング定義ファイル）に新しいパスを登録します。

```
// 例：router/index.js または該当のルーティングファイル

import AdminUserListView from '../views/AdminUserListView.vue'; // パスは適宜合わせてください

const routes = [
  // ... 既存のルート定義 (/dashboard や /account-edit など)

  {
    path: '/admin', // ★ハンバーガーメニューの「管理者画面」で指定したパス
    name: 'AdminUserList',
    component: AdminUserListView
  }
];
```

本機能の最終確認シナリオ

すべてのソースコードの保存が完了したら、システムを立ち上げて、仕様が美しく連動する感動のループをテストしてみましょう！

- **一般ユーザーで退会申請を出す**

- 一般ユーザー（例： `test@example.com` など）でログインし、先ほど実装した「アカウント情報編集画面」へ向かいます。
- 「アカウント削除申請をする」ボタンを押し、2段階の確認を「OK」で通過させ、自動ログアウトされるのを待ちます。

- **統括管理者でログインする**

- ログアウト後、起動時に自動投入された管理者アカウント ** `admin@example.com` / `admin1234**` でログインします。

- **管理者画面へアクセスする**

- ハンバーガーメニューを開くと、見事に  **管理者画面** が出現しています。そこから管理者画面へ遷移します。

- **バッジと承認のテスト**

- アカウント一覧表の中に、先ほど申請を出した一般ユーザーの行が**薄黄色**になり、「**退会申請中**」という**バッジ**がピコピコと点滅しているのを確認します。
- その横にある「退会を承認する」ボタンを押し、最終アラートで「OK」を選択します。
- 画面上に緑色で「～退会承認が完了しました」と表示され、一覧表からそのユーザーのレコードが綺麗サッパリ消滅（物理削除）することを確認します！

ユーザー側の「申請」から、管理者側の「視覚的バッジ検知」「物理削除による承認」までの一連の大きなサイクルがこれで繋がります。

フロントエンドのバージョンアップ

ここからは、先程まで作成してきた「管理者用：アカウント一覧・退会管理」画面に、テーブルのページネーション、複数ワード検索、動的ソート（昇順・降順・退会申請優先など）、表示件数切り替えなどの機能を盛り込んでいきます。

これらの機能は、管理対象のデータが増えてきた際に画面が重くなるのを防ぎ、かつ目的のデータを一瞬で見つけ出すべく、実務のWebアプリでも「必須」とされる極めて重要なものといえます。

今回は、バックエンド（Java）からデータを全件取得する既存のシンプルな仕組み

（`ApiClient.get('/users/admin/list')`）をそのまま活かしつつ、Vue.js（フロントエンド）側のリアクティブな計算（`computed`）を駆使して、すべての検索・ソート・ページネーションを完全リアルタイム高速処理する「高機能データテーブル仕様」へと進化させるソースコードをご提示します。

これを行えば、サーバーサイドのコードを余計に複雑にすることなく、初学者にも理解しやすい形のまま、極めてレスポンスで快適な管理者UIが実現できます。

AdminUserListView.vue のアップデート

既存の `AdminUserListView.vue` を、以下のコードに丸ごと置き換えてください。UIデザインを損なうことなく、検索バー、ソートヘッダー、ページネーションコントロールを美しく統合しています。

```
<template>
  <div class="admin-container">
    <div class="admin-header-box">
      <h2>管理者用：アカウント一覧・退会管理</h2>
      <p class="admin-subtitle">登録されているユーザーの確認、検索・ソート、および退会申請の承認を行えます。</p>
    </div>

    <div v-if="isLoading" class="loading-box">
      データを読み込み中...
    </div>

    <div v-if="errorMessage" class="alert alert-danger">{{ errorMessage }}</div>
    <div v-if="successMessage" class="alert alert-success">{{ successMessage }}</div>
  </div>
</template>
```



```

<div v-if="!isLoading && filteredAndSortedUsers.length > 0" class="table-responsive">
  <table class="user-table">
    <thead>
      <tr>
        <th @click="toggleSort('userName')" class="sortable-header">
          ユーザー名 {{ getSortIcon('userName') }}
        </th>
        <th @click="toggleSort('userId')" class="sortable-header">
          ユーザーID (メールアドレス) {{ getSortIcon('userId') }}
        </th>
        <th @click="toggleSort('isAuth')" class="sortable-header">
          権限 {{ getSortIcon('isAuth') }}
        </th>
        <th @click="toggleSort('quitDemand')" class="sortable-header">
          ステータス {{ getSortIcon('quitDemand') }}
        </th>
        <th>操作</th>
      </tr>
    </thead>
    <tbody>
      <tr v-for="user in paginatedUsers" :key="user.accountId" :class="{ 'row-quit-pending': user.quitDemand === 1 }">
        <td><strong>{{ user.userName }}</strong></td>
        <td>{{ user.userId }}</td>
        <td>
          <span :class="['badge', user.isAuth === 1 ? 'badge-admin' : 'badge-general']">
            {{ user.isAuth === 1 ? '統括管理者' : '一般ユーザー' }}
          </span>
        </td>
      </tr>
    </tbody>
  </table>
</div>

```

```
<td>
  <span v-if="user.quitDemand === 1" class="badge badge-warning animate-pulse">
    ⚠ 退会申請中
  </span>
  <span v-else class="badge badge-normal">
    正常稼働
  </span>
</td>
<td>
  <button
    v-if="user.quitDemand === 1 && user.userId !== 'admin@example.com'"
    @click="handleApproveQuit(user)"
    class="btn-approve"
  >
    退会を承認する
  </button>
  <span v-else-if="user.userId === 'admin@example.com'" class="text-muted-info">-</span>
  <span v-else class="text-muted">-</span>
</td>
</tr>
</tbody>
</table>
</div>
```

```
<div v-if="!isLoading && filteredAndSortedUsers.length > 0" class="pagination-panel">
  <div class="pagination-info">
    全 {{ filteredAndSortedUsers.length }} 件中 {{ startIndex + 1 }} ~ {{ endIndex }} 件目を表示
  </div>
  <div class="pagination-buttons">
    <button :disabled="currentPage === 1" @click="currentPage--" class="btn-page">◀ 前へ</button>
    <button
      v-for="page in totalPages"
      :key="page"
      @click="currentPage = page"
      :class="['btn-page-number', { 'active': currentPage === page }]"
    >
      {{ page }}
    </button>
    <button :disabled="currentPage === totalPages" @click="currentPage++" class="btn-page">次へ ▶</button>
  </div>
</div>

<div v-else-if="!isLoading" class="empty-box">
  条件に一致するアカウントが存在しません。
</div>
</div>
</template>
```

```
<script setup>
import { ref, onMounted, computed, watch } from 'vue';
import { useRouter } from 'vue-router';
import apiClient from '../api';

const router = useRouter();

const users = ref([]);
const isLoading = ref(true);
const errorMessage = ref('');
const successMessage = ref('');

// 💡 状態管理用のリアクティブ変数
const searchQuery = ref(''); // 検索窓の文字列
const perPage = ref(10); // 1ページあたりの表示件数 [10, 50, 100, 500]
const currentPage = ref(1); // 現在のページ番号
const sortKey = ref('quitDemand'); // 初期ソートは「退会申請」を優先させるためのキー
const sortOrder = ref('desc'); // 1（申請中）が上に来るように初期は降順（desc）

// データ取得ロジック（変更なし）
const fetchUserList = async () => {
  try {
    errorMessage.value = '';
    const response = await apiClient.get('/users/admin/list');
    users.value = response.data;
  } catch (error) {
    console.error(error);
    errorMessage.value = 'アカウント一覧の取得に失敗しました。';
  } finally {
    isLoading.value = false;
  }
};
```

```

onMounted(() => {
  const userData = localStorage.getItem('user');
  if (userData) {
    const user = JSON.parse(userData);
    if (user.isAuth !== 1) {
      alert('この画面は管理者専用です。');
      router.push('/dashboard');
      return;
    }
    fetchUserList();
  } else {
    router.push('/login');
  }
});

// 💡 核心ロジック1: 【検索 & ソート】を動的に同時計算するロジック
const filteredAndSortedUsers = computed(() => {
  let result = [...users.value];

  // 【仕様通り: 半角スペース区切りの複数ワード検索】
  if (searchQuery.value.trim()) {
    // 全角スペースがあれば半角スペースに変換し、配列に分解
    const keywords = searchQuery.value.replace(/ /g, ' ').toLowerCase().split(' ').filter(w => w);

    result = result.filter(user => {
      // ユーザー名とユーザーID（メールアドレス）を検索対象にする
      const targetText = `${user.userName} ${user.userId}`.toLowerCase();
      // 分解したキーワード「すべて」が含まれている行だけ残す（AND検索）
      return keywords.every(keyword => targetText.includes(keyword));
    });
  }

  // 【動的ソート処理】
  result.sort((a, b) => {
    let modifier = sortOrder.value === 'desc' ? -1 : 1;

    // 文字列または数値の比較
    let valA = a[sortKey.value];
    let valB = b[sortKey.value];

    // 文字列の場合は大文字小文字を区別せず比較
    if (typeof valA === 'string') valA = valA.toLowerCase();
    if (typeof valB === 'string') valB = valB.toLowerCase();

    if (valA < valB) return -1 * modifier;
    if (valA > valB) return 1 * modifier;
    return 0;
  });

  return result;
});

```

```
// 💡 核心ロジック2: 【ページネーション】現在のページに該当するデータだけを切り出すロジック
const totalPages = computed(() => {
  return Math.ceil(filteredAndSortedUsers.value.length / perPage.value) || 1;
});

const startIndex = computed(() => (currentPage.value - 1) * perPage.value);
const endIndex = computed(() => {
  const end = startIndex.value + perPage.value;
  return end > filteredAndSortedUsers.value.length ? filteredAndSortedUsers.value.length : end;
});

const paginatedUsers = computed(() => {
  return filteredAndSortedUsers.value.slice(startIndex.value, endIndex.value);
});

// 検索条件や表示件数が変わったら、ページ番号を強制的に1ページ目に戻す親切設計
watch([searchQuery, perPage], () => {
  currentPage.value = 1;
});

// ソートキーを切り替える関数
const toggleSort = (key) => {
  if (sortKey.value === key) {
    // 同じヘッダーが叩かれたら 昇順 ⇄ 降順 を反転
    sortOrder.value = sortOrder.value === 'asc' ? 'desc' : 'asc';
  } else {
    // 新しいヘッダーなら昇順からスタート
    sortKey.value = key;
    sortOrder.value = 'asc';
  }
};
```



```
// ソート状態を視覚的に表現するアイコン
const getSortIcon = (key) => {
  if (sortKey.value !== key) return '🔍';
  return sortOrder.value === 'asc' ? '▲' : '▼';
};

// 退会承認処理（変更なし）
const handleApproveQuit = async (targetUser) => {
  successMessage.value = '';
  errorMessage.value = '';

  const confirmAction = confirm(`【最終警告】 \n本当に「${targetUser.userName}」さんの退会申請を承認しますか？\nこの操作を行うと、アカウント情報はシステムから完全に物理削除されます。`);
  if (!confirmAction) return;

  try {
    await apiClient.delete(`/users/approve-quit/${targetUser.accountId}`);
    successMessage.value = `「${targetUser.userName}」さんのアカウント削除（退会承認）が完了しました。`;
    await fetchUserList();
  } catch (error) {
    if (error.response && error.response.data) {
      errorMessage.value = error.response.data;
    } else {
      errorMessage.value = '承認処理の通信中にエラーが発生しました。';
    }
  }
};
</script>
```

```
<style scoped>
.admin-container {
  max-width: 1000px;
  margin: 30px auto;
  padding: 20px;
}
.admin-header-box {
  background: #ffffff;
  padding: 20px;
  border-radius: 8px;
  box-shadow: 0 2px 8px rgba(0,0,0,0.05);
  margin-bottom: 25px;
  border-left: 5px solid #dc3545;
}
h2 { margin: 0 0 5px 0; color: #333; }
.admin-subtitle { margin: 0; color: #6c757d; font-size: 0.95rem; }

/* 🟡 追加：コントロールパネルのスタイル */
.control-panel {
  display: flex;
  gap: 20px;
  background: #ffffff;
  padding: 15px 20px;
  border-radius: 8px;
  box-shadow: 0 2px 6px rgba(0,0,0,0.05);
  margin-bottom: 15px;
  align-items: flex-end;
}
.search-box { flex: 1; }
.per-page-box { width: 120px; }
.control-label {
  display: block;
  font-size: 0.85rem;
  font-weight: bold;
  color: #495057;
  margin-bottom: 5px;
}
.input-search, .select-per-page {
  width: 100%;
  padding: 8px 12px;
  border: 1px solid #ced4da;
  border-radius: 4px;
  font-size: 0.9rem;
  box-sizing: border-box;
}

/* 追加：ソート可能なヘッダーのカーソル演出 */
.sortable-header {
  cursor: pointer;
  user-select: none;
  transition: background-color 0.2s;
}
.sortable-header:hover {
  background-color: #e9ecef !important;
}
```

```

/* テーブルデザイン */
.table-responsive {
  background: #ffffff;
  border-radius: 8px;
  box-shadow: 0 4px 12px rgba(0,0,0,0.08);
  overflow: hidden;
}
.user-table { width: 100%; border-collapse: collapse; text-align: left; }
.user-table th, .user-table td { padding: 15px 20px; border-bottom: 1px solid #dee2e6; }
.user-table th { background-color: #f8f9fa; color: #495057; font-weight: bold; }
.row-quit-pending { background-color: #fffdf5; }

/* バッジとアニメーション */
.badge { display: inline-block; padding: 5px 10px; font-size: 0.8rem; font-weight: bold; border-radius: 20px; }
.badge-admin { background-color: #e3f2fd; color: #0d47a1; }
.badge-general { background-color: #e8f5e9; color: #1b5e20; }
.badge-normal { background-color: #f1f3f5; color: #6c757d; }
.badge-warning { background-color: #fff3cd; color: #856404; border: 1px solid #ffeeba; }

@keyframes pulse { 0% { opacity: 1; } 50% { opacity: 0.6; } 100% { opacity: 1; } }
.animate-pulse { animation: pulse 2s infinite; }

.btn-approve {
  background-color: #dc3545; color: white; border: none; padding: 8px 14px;
  border-radius: 4px; font-size: 0.85rem; font-weight: bold; cursor: pointer;
  transition: background-color 0.15s;
}
.btn-approve:hover { background-color: #bd2130; }
.text-muted { color: #ced4da; }
.text-muted-info { color: #adb5bd; font-size: 0.85rem; font-style: italic; }

/* 🍷 追加：ページネーションパネルのスタイル */
.pagination-panel {
  display: flex;
  justify-content: space-between;
  align-items: center;
  margin-top: 20px;
  background: #ffffff;
  padding: 12px 20px;
  border-radius: 8px;
  box-shadow: 0 2px 6px rgba(0,0,0,0.05);
}
.pagination-info { font-size: 0.9rem; color: #495057; }
.pagination-buttons { display: flex; gap: 5px; }
.btn-page {
  padding: 6px 12px; background-color: #ffffff; border: 1px solid #ced4da;
  border-radius: 4px; font-size: 0.85rem; cursor: pointer; transition: all 0.2s;
}
.btn-page:hover:not(:disabled) { background-color: #f8f9fa; border-color: #b5bbc1; }
.btn-page:disabled { color: #ced4da; cursor: not-allowed; }

.btn-page-number {
  padding: 6px 12px; background-color: #ffffff; border: 1px solid #ced4da;
  border-radius: 4px; font-size: 0.85rem; cursor: pointer;
}
.btn-page-number:hover { background-color: #f8f9fa; }
.btn-page-number.active {
  background-color: #007bff; color: white; border-color: #007bff; font-weight: bold;
}

.alert { padding: 15px; margin-bottom: 20px; border-radius: 4px; font-weight: bold; }
.alert-danger { background-color: #f8d7da; color: #721c24; border-left: 5px solid #dc3545; }
.alert-success { background-color: #d4edda; color: #155724; border-left: 5px solid #28a745; }
.loading-box, .empty-box { text-align: center; padding: 40px; color: #6c757d; font-size: 1.1rem; }
</style>

```

改良ポイントと解説

1. 複数ワードAND検索の実現（仕様準拠）

検索窓に入力された文字列を、スペース区切り（全角・半角双方を考慮）でキーワードの配列に分解します。名前とIDのどちらかに、入力された「すべてのキーワード」が含まれている行だけを抽出する、本格的な検索ロジックです。

2. テーブルヘッダーのクリックによる動的ソート

各列のヘッダーをクリックすると、その項目を基準に昇順・降順がトグルで切り替わります。初期状態では、一番気づくべき「退会申請（`quitDemand`）」の降順（＝申請中のユーザーが一番上に集まる状態）をデフォルトに設定しており、管理者が一目で仕事を見つけられる実用的な設計です。

3. シームレスなページネーション

表示件数を「10件」から「500件」などに切り替えると、自動的にトータルのページ数が再計算されます。また、「検索ワードを打ち込んだら、該当データが存在するはずなのに、現在のページが2ページのままで何も映らない」というページネーション特有のバグを防ぐため、`watch` を使って検索時は自動的に1ページ目へ引き戻す親切設計が施されています。